

HW4 Model Validation Report

Group 4 沈伯衡 李雨佳 傅恒恒

2025 年 11 月 19 日

摘要

本报告对 Group 5 的代码和模型进行了全面检查，重点关注可复现性、数据处理流程与样本划分策略三方面。整体来看，模型思路清晰，但在外部文件管理、特征工程的重复、复杂处理，以及训练和验证样本的划分方式上，仍有一些会影响模型稳定性和可靠性的问题。针对这些情况，我们提出了五条改进建议：统一管理所有外部文件以保证流程可复现；搭建一个统一的特征工程 Pipeline 避免反复处理同一字段；针对 TF-IDF 的改进建议；基于其他方法的地理位置特征处理方法；以及将样本划分提前、确保预处理只在训练集上进行，从而避免信息泄漏。这些建议有助于提高模型的透明度、一致性，也让后续的调参和扩展更顺畅。

1 模型评估

1.1 可复现性

文件路径依赖本地，无法复现 代码中出现多次出现“rent_predicted_updated.csv”，但由于并未附上该文件，导致后续无法复现

```
> ~
import pandas as pd
import os

base_file = 'ruc_Class25Q2_test_price_predicted.csv'
src_file = 'rent_predicted_updated.csv'

# 1. 读取基底数据 (优先用 submit1027.csv, 没有就再看目标文件是否存在)
dst_file = 'submit1027.csv'
if os.path.exists(base_file):
    base = pd.read_csv(base_file)
else:
    base = pd.DataFrame(columns=['ID', '预测价格']) # 空表头, 确保后面 concat 不出错

# 2. 读取待追加的 ID/Price
new = pd.read_csv(src_file, usecols=['ID', 'Price'])
new.columns = ['ID', '预测价格'] # 统一列名

# 3. 拼接
out = pd.concat([base, new], axis=0, ignore_index=True)

# 4. 写出
out.to_csv(dst_file, index=False)
print(f'已完成: 共 {len(out)} 条记录 -> {dst_file}')

[62] 0.4s Python
```

```
FileNotFoundError                                Traceback (most recent call last)
Cell In[62], line 15
     12 base = pd.DataFrame(columns=['ID', '预测价格']) # 空表头, 确保后面 concat 不出错
     14 # 2. 读取待追加的 ID/Price
--> 15 new = pd.read_csv(src_file, usecols=['ID', 'Price'])
     16 new.columns = ['ID', '预测价格'] # 统一列名
     18 # 3 拼接
```

图 1: 并无该文件，后续无法复现

此外，代码未提供 requirements.txt 或环境版本说明，对于跨环境复现完全一致结果仍存在潜在风险；另外，文本处理部分依赖词向量生成，如果未来尝试使用中文分词器（如 jieba），词典版本可能影响最终结果。建议补充依赖版本与运行环境说明，以进一步提高跨设备复现能力。

1.2 Data Processing Validation

该代码对数据进行了较为全面的预处理，包括：

文本特征处理 将多字段文本拼接后进行 TF-IDF 表示，并通过 SVD 进行降维，有效将高维稀疏文本矩阵转换为低维连续特征，减少过拟合风险。

类别变量编码 对诸如楼层、房屋朝向、房屋用途等离散变量进行了 One-hot 编码，并对城市、区域、板块等地理类变量进行了进一步编码，使模型能够利用分类特征。

缺失值处理与类型转换 对部分数值及分类特征进行了缺失值填补，避免模型训练阶段出错；字符串型数值（如“120 m²”）被转换为连续数值，使模型能够直接利用其量纲含义。整体数据处理思路较为严谨，能确保模型在结构化与文本混合数据上正常训练。但也存在如下问题：

多次对相同字段做 get dummies 填充，流程复杂度非常高 对“楼层、建筑类型、区域”等字段，多次补空值、重命名、再 dummy，一部分逻辑在前面，另一部分在后面，又有专门的 target_villa_types 处理。这类重复执行的处理，加入 Notebook 执行顺序不同，会导致特征数不同，进而导致模型不同。以下是一小部分示例：

```
# 1. 生成所有类别的独热编码（共6个虚拟变量）
# prefix指定虚拟变量的前缀，方便区分
floor = pd.get_dummies(df['所在楼层'], prefix='楼层', drop_first=True, dtype=int) #+U 添加上下文到对话

# 3. 删除原“所在楼层”列，合并虚拟变量到原数据框
df = df.drop(columns=['所在楼层']).join(floor)

floort = pd.get_dummies(df_test['所在楼层'], prefix='楼层', drop_first=True, dtype=int)
```

图 2

```
# 生成虚拟变量：为每个权属类型创建0/1列（prefix指定前缀，方便识别）
dumfft = pd.get_dummies(df_test['房屋用途'], prefix='用途', drop_first=True, dtype=int) #+U 添加上下文到对话
```

图 3

```
# 生成虚拟变量：为每个权属类型创建0/1列（prefix指定前缀，方便识别）
dummies = pd.get_dummies(df['交易权属'], prefix='权属', drop_first=True, dtype=int) #+U 添加上下文到对话

# 删除原始“交易权属”列，合并虚拟变量到原DataFrame
```

图 4

```
# ----- 原有虚拟变量生成逻辑（不变） -----
# 生成虚拟变量：为每个权属类型创建0/1列（prefix指定前缀，方便识别）
dumff = pd.get_dummies(df['房屋用途'], prefix='用途', drop_first=True, dtype=int) #+U 添加上下文到对话
```

图 5

One-hot 后大量新增列 该部分代码对城市、区域、板块变量做了 one-hot 编码，但这是有较大风险的，尤其是板块数据最后被划分为了 1186 列。首先，因为会产生大量的新列，导致维度大幅增加，加大模型训练难度和时间。其次，数据可能过于稀疏，对测试集可能会出现新板块会全为 0。模型容易记住训练集具体位置，而不是学习其中的规律，容易导致过拟合现象。

```
# 分类变量列名（需要转为虚拟变量）
categorical_cols = ["城市", "区域", "板块"]
# 连续变量列名（直接保留原始值）
continuous_cols = ["建筑面积", "lon", "lat"]
binary_cols = ["is_共有"]
# 1. 提取已有的虚拟变量（楼层、权属开头的列）
existing_dummies = df.filter(regex='^楼层_|^权属_|^用途_|^别墅_|^朝向_')
```

图 6

```
'区域_99.0', '区域_101.0', '区域_102.0', '区域_103.0', '区域_104.0', '区域_105.0', '区域_106.0',
'区域_107.0', '区域_108.0', '区域_109.0', '区域_110.0', '区域_111.0', '区域_112.0', '区域_113.0',
'区域_114.0', '区域_115.0', '区域_117.0', '区域_118.0', '区域_119.0', '区域_120.0', '区域_121.0',
'区域_122.0', '区域_123.0', '区域_124.0', '区域_125.0', '区域_126.0', '区域_127.0', '区域_128.0',
'区域_129.0', '区域_130.0', '区域_131.0', '板块_2.0', '板块_3.0', '板块_4.0', '板块_5.0', '板块_6.
0', '板块_8.0', '板块_9.0', '板块_12.0', '板块_13.0', '板块_14.0', '板块_15.0', '板块_16.0', '板块
_17.0', '板块_19.0', '板块_20.0', '板块_21.0', '板块_22.0', '板块_24.0', '板块_25.0', '板块_26.
0', '板块_27.0', '板块_28.0', '板块_29.0', '板块_30.0', '板块_33.0', '板块_34.0', '板块_35.0', '板
块_36.0', '板块_37.0', '板块_38.0', '板块_39.0', '板块_40.0', '板块_41.0', '板块_42.0', '板块_43.
0', '板块_44.0', '板块_45.0', '板块_46.0', '板块_48.0', '板块_49.0', '板块_50.0', '板块_51.0', '板
块_53.0', '板块_55.0', '板块_56.0', '板块_57.0', '板块_58.0', '板块_59.0', '板块_60.0', '板块_61.
0', '板块_62.0', '板块_63.0', '板块_64.0', '板块_65.0', '板块_66.0', '板块_67.0', '板块_68.0', '板
块_69.0', '板块_70.0', '板块_71.0', '板块_72.0', '板块_73.0', '板块_74.0', '板块_75.0', '板块_76.
0', '板块_78.0', '板块_79.0', '板块_80.0', '板块_82.0', '板块_83.0', '板块_84.0', '板块_85.0', '板
块_86.0', '板块_87.0', '板块_88.0', '板块_89.0', '板块_90.0', '板块_92.0', '板块_93.0', '板块_97.
0', '板块_98.0', '板块_100.0', '板块_101.0', '板块_102.0', '板块_105.0', '板块_106.0', '板块_107.
```

图 7: 产生大量的新列，维度大幅增加

1.3 Sample choice

- 代码直接读取预先拆分好的 df（训练集）与 df_test（测试集），训练过程中未将测试集数据参与模型拟合，因此不存在明显的泄露问题。同时，特征工程函数对训练与测试分别调用 fit_transform 与 transform，避免了“使用测试数据进行特征抽取”的情况。

- 虽然训练集与测试集已分开，但整个训练集内部未再划分验证集，因此模型调参及性能评估完全依赖测试集，这可能导致测试集在迭代中被“当作验证集使用”，影响评估公平性。建议在训练集内部加入 train/valid split 或 K-fold CV，以便在不泄露测试集信息的前提下更稳定地选择模型与超参数。

2 Suggestions

2.1 统一外部文件管理，保障完整可复现性

建议将所有外部文件进行统一管理，明确数据文件的存放路径、命名规则，并提供相应的执行脚本。同时附上复现清单，例如数据来源、代码版本与关键参数设置等。这样可以确保其他人在不同的环境中也能完整、顺利地复现模型的全部结果。

2.2 构建统一的特征工程 Pipeline，避免重复处理

建议使用统一的特征工程 Pipeline（如 sklearn 的 ColumnTransformer 或 Pipeline），把缺失值处理、编码、文本特征提取等步骤集中封装，并仅在训练集上进行 fit。这样可以确保每个字段只被处理一次，避免 notebook 多个 cell 的执行顺序对特征维度造成影响，同时保证测试集按照完全一致的流程进行 transform，避免 train 与 test 之间出现特征不对齐的问题。

这样做可以确保每个字段只被处理一次，避免 notebook 多 cell 执行顺序造成特征维度随机波动。同时可确保测试集严格执行 transform，不再出现 df 与 df_test 分别 dummy 造成的特征不一致问题。

2.3 针对 TF-IDF 的改进建议

```
# TF-IDF 向量化（将文本转为数值矩阵）
vect = TfidfVectorizer(
    ngram_range=(1, 2), # 考虑单个词和词组（1-2个词）
    max_features=cfg.TEXT_MAX_FEATURES,
    min_df=cfg.TEXT_MIN_DF
)
Xtr_txt = vect.fit_transform(tr_docs) # 训练集拟合并转换
n_feat = Xtr_txt.shape[1] # 词汇量（特征数）
```

图 8

针对 TF-IDF 这部分有如下改进建议：

【引入 jieba 等适用于中文的分词工具】：该部分对文本进行向量化使用了 TfidfVectorizer 方法。然而该方法默认采用英文式分词规则，即按照空格、标点与连续字母进行切分。由于中文文本不存在天然的空格分词结构，该方法无法有效识别中文词语，往往将整段文本视为单个 token，导致生成的文本向量缺乏语义信息，从而使文本 embedding 失真。因此，应引入适用于中文的分词工具（如 jieba、pkuseg 等），在向量化前对文本进行分词处理，以提取有效语义特征并提升模型预测表现。

【尝试更能够识别文本语境的模型】：当前使用 TF-IDF 构造文本特征，但 TF-IDF 基于

词频统计，不具备语义理解能力，也无法处理同义表达。后续可考虑引入 Word2vec 或基于预训练语言模型的句向量（如 BERT），以提升文本特征表达能力。后两个模型也是当前自然语言处理中使用较多的模型。

【可尝试分列编码而非拼接】：不同字段语义不同，可以给每列单独做 TF-IDF，再拼合。这样可以让每类文本独立降维，信息损失更少。

字段	应捕获的信息
核心卖点	房源价值亮点
户型介绍	结构与功能描述
周边配套	设施条件
房屋优势/客户反馈	市场主观评价

2.4 基于其他方法的地理位置特征处理方法的改进建议

针对地理位置变量可按如下方式处理：

【可按照经纬度进行聚类】：基于经纬度聚类，可以把地段分成 20 类，比板块 one-hot 泛化性强。

【Target encoding】：可以用每个板块用“平均房价”替代 one-hot。

【按城市分别进行模型训练与预测】：可尝试使用城市进行分类，对不同地区的房价和租金进行单独的预测。

2.5 调整样本划分流程：先划分，再预处理

建议将样本划分提前进行：首先将原始数据划分为训练集与验证集，然后仅在训练集上拟合所有预处理步骤，再将相同的处理流程应用到验证集与最终测试集。此方法可以避免特征工程阶段对验证集的信息泄漏，使验证误差更加真实地反映模型的泛化能力。